Building a Full-Stack Business Application in 2 Days with AI-Assisted Development

How Claude Code helped deliver a production-ready dairy farm management system — from zero to deployed — in a single weekend.

The Challenge

A dairy processing business needed a digital system to replace manual registers and spreadsheets. The requirements were substantial:

- Track 4 production flows (milk processing, cream-to-butter/ghee, butter-to-ghee, dahi production)
- Record purchases from vendors with SNF/Fat quality metrics
- Manage sales to customers across multiple products
- Maintain a 30-day running stock ledger with cumulative balances
- Generate financial reports: sales reports, vendor purchase reports, stock valuation
- Support multi-location operations with role-based access control
- Provide audit logging of every data change
- Detect production anomalies automatically
- Track vendor payments and ledger balances

This isn't a toy app. It's a multi-user business system with real money flowing through it.

The Stack

The architecture was deliberately pragmatic — chosen to minimise infrastructure complexity while maximising reach:

Layer	Technology	Why
Frontend	Flutter (Web + Mobile)	Single codebase for all platforms
Backend	WordPress REST API (PHP plugin)	Existing hosting, zero DevOps overhead
Database	MariaDB	Comes with WordPress, battle-tested

Auth	JWT tokens	Stateless, standard
Hosting	AWS (Bitnami WordPress)	Already provisioned

A Flutter frontend talking to a WordPress plugin backend isn't a textbook architecture — but it's exactly the right one for this context. The business already had a WordPress site. Adding a custom REST API plugin meant zero additional infrastructure, zero additional cost, and a deployment workflow as simple as `scp` a single PHP file.

What Got Built

By the Numbers

Metric	Count
Dart source files	40+
Dart lines of code	7,500+
PHP lines of code	3,150+
REST API endpoints	35
Database tables	16
Production flows	4
Fixed product types	9
Report types	8
Unit & integration tests	52
Static analysis issues	0
Git commits to completion	8
Calendar time (first commit to deployed)	~32 hours

Feature Inventory

Data Entry

- Full-fat milk purchase with vendor, SNF, Fat, rate
- Milk processing (FF Milk to Skim Milk + Cream)
- Cream purchase and processing (to Butter + Ghee)
- Butter purchase and processing (to Ghee)
- Dahi production (SMP + Protein + Culture + Skim Milk to containers)
- Ingredient purchases (SMP, Protein, Culture)

- Sales entry with customer, product, quantity, rate

Reports & Analytics

- 30-day running stock with cumulative daily balances
- Sales report pivoted by product (KG + value)
- Vendor purchase history
- Stock valuation with estimated rates (finance role only)
- Production and sales transaction logs
- Production anomaly detection (yield ratio outliers)
- Vendor ledger with payment tracking
- Funds report

Platform Capabilities

- JWT authentication with auto-refresh
- Multi-location support with location-scoped data
- Role-based access (operator vs. finance)
- Offline detection with banner
- Audit trail on every data mutation
- Self-healing database migrations
- Responsive web layout constrained to mobile form factor
- CSV export on all reports (browser download on web, share sheet on mobile)

The AI-Assisted SDLC

Here's what the development lifecycle actually looked like, and where AI changed the game.

1. Requirements & Architecture (Hour 0-1)

The process started with a conversation. I described the business domain — production flows, stock logic, access control rules — and Claude Code helped structure it into a formal `CLAUDE.md` specification. This single file became the project's source of truth: database schema, API contracts, Flutter conventions, deployment procedures, and known gotchas.

What AI changed: Traditionally, writing a specification this detailed takes days of back-and-forth. With AI, the domain knowledge in my head was extracted, structured, and formalised in under an hour. The spec wasn't just documentation — it became executable context that guided every subsequent code generation.

2. Database Design (Hour 1-2)

16 tables with proper constraints, indexes, foreign keys, and a self-healing migration system. The schema had to handle nuances like:

- Decimal precision for financial calculations (`DECIMAL(10,2)`)
- Composite unique constraints to prevent duplicate sales entries
- A user flags table where `user_id` is the primary key (no auto-increment `id`)
- Cascade deletes on user-location access mappings

What AI changed: Schema design is where subtle bugs are born. AI caught type mismatches early (culture/protein needed `DECIMAL(10,2)` not `INT`) and maintained consistency between PHP column references and Dart model types across the full stack.

3. Backend API Development (Hours 2-8)

2,300+ lines of PHP in a single WordPress plugin file. 35 endpoints following a consistent pattern: validate inputs, check location access, execute query, audit log, return standardised JSON envelope.

The plugin included:

- A helper method library (`d1()`, `d2()`, `ok()`, `err()`, `audit()`)
- A reusable route registration wrapper
- Self-healing DB migrations that check `information_schema` before altering
- Complex stock calculations aggregating across 6 tables with cumulative running totals

What AI changed: The consistency is the remarkable part. Every endpoint follows the same error handling pattern. Every response uses the same JSON envelope. Every mutation writes an audit log entry. Maintaining this discipline across 35 endpoints is exactly the kind of thing humans get sloppy about on endpoint #27. AI doesn't get tired.

4. Frontend Development (Hours 4-16)

6,700+ lines of Dart across 37 files. The architecture is clean MVC with GetX:

- **Models** — shared DTOs matching the API response shapes
- **Controllers** — one per feature, handling API calls and state
- **Pages** — reactive UI bound to controller observables
- **Core services** — auth, API client, permissions, connectivity, location, navigation

A shared widget library (`shared_widgets.dart`) enforced visual consistency: `IntField`, `DecimalKgField`, `SnfFatField`, `RateField`, `DCard`, `FeedbackBanner`, etc.

What AI changed: The frontend-backend contract is where full-stack projects typically leak time. Field names, types, endpoint paths, response structures — any mismatch means a bug that only surfaces at runtime. With AI holding both sides of the contract in context (via `CLAUDE.md`), these mismatches were caught at generation time, not debug time.

5. Testing & Bug Fixing (Hours 16-28)

Testing was done interactively: run on Chrome, exercise each flow, observe console logs, fix issues in real-time. The AI could:

- Read error messages from the browser console
- Trace them to the specific PHP endpoint or Dart controller
- Propose and apply fixes
- Re-run and verify

The system is battle-tested with **52 unit and integration tests** across 12 test files, covering all controllers (production, sales, stock, reports, vendor ledger, anomaly detection, funds, transactions) and the UI navigation layer. A `FakeApiClient` pattern enables deterministic controller testing without network calls.

What AI changed: The debug cycle compressed dramatically. Instead of: read error → search codebase → understand context → write fix → test — the cycle became: read error → AI already has context → fix applied → test. A bug that might take 30 minutes to diagnose and fix took 2-3 minutes.

6. Deployment (Hours 28-32)

- PHP plugin: `scp` to the server — live immediately
- Flutter web: `flutter build web --base-href /dairyapp/` → upload to server
- Database: migrations run automatically on first request

What AI changed: Deployment scripts and procedures were generated alongside the code. The AI knew the server paths, the PEM key location, the build flags. No scrambling to remember deployment steps.

What Worked Exceptionally Well

The Living Specification

`CLAUDE.md` isn't just a README. It's a 400+ line specification that the AI reads at the start of every session. It contains the database schema, API contracts, widget inventory, deployment procedures, and known gotchas. When a new feature is

added, the spec is updated. When a bug reveals a subtle constraint (like `SaleEntry` must live in `sales_controller.dart`, not `models.dart`), it goes into the "Known Gotchas" section.

This created a flywheel: the more we built, the better the spec got, and the better the spec got, the fewer bugs the AI introduced.

Full-Stack Context

The killer advantage of AI-assisted development on this project was holding both sides of the stack in context simultaneously. When I said "add vendor ledger tracking," the AI generated:

- The database table
- The PHP endpoint with proper access control and audit logging
- The Dart controller with API calls
- The Flutter page with proper widgets
- Updates to the route registration
- Updates to the CLAUDE.md spec

All consistent. All type-safe across the PHP-Dart boundary. All following established patterns.

Pattern Consistency at Scale

By the 30th endpoint, the code was just as disciplined as the 1st. Same error handling. Same audit logging. Same JSON envelope. Same input validation. Humans drift. AI follows the pattern.

Gaps and Honest Assessment

No project ships perfect, and intellectual honesty requires acknowledging what's missing. Here's what a mature version of this system would add — these are deliberate trade-offs, not oversights:

What Was Closed

- **Testing:** Went from 4 test files to 12, with 52 unit and integration tests covering all controllers and UI navigation. Zero static analysis issues.
- **Security audit:** Full review completed — all SQL uses `wpdb::prepare()`, no CRITICAL/HIGH findings. MEDIUM items (type-cast precision, date validation) are tracked.
- **CSV export:** All 8 reports now support CSV download (browser file download on web, native share sheet on mobile via `share_plus`).

- **Code quality:** Flutter analyzer passes with zero warnings. All deprecated API calls (`withOpacity`) replaced with modern equivalents (`withValues`).

Known Trade-offs (Backlog)

Security

- Rate limiting on API endpoints (WordPress-level plugin available)
- Type-cast precision: `CAST(AS SIGNED)` truncates `DECIMAL(10,2)` for protein/culture in stock queries
- Date format validation on POST handlers

Operational

- No CI/CD pipeline — deployment is manual `scp`
- No automated backups strategy documented
- No monitoring or alerting — if the API goes down, nobody knows until a user reports it
- No error tracking (Sentry, Bugsnag, etc.)

UX

- No offline data entry — requires connectivity for every operation
- No bulk operations — each entry is saved individually
- No undo/edit on production entries — only sales can be deleted

Architecture

- Single PHP file — 3,150+ lines in one file works but will become unwieldy
- No API versioning beyond the namespace
- WordPress coupling — the business logic is embedded in a WordPress plugin

---

Lessons Learned

1. The Spec Is the Product

The single most valuable artifact in this project isn't the code — it's `CLAUDE.md`. It's the institutional memory that survives across sessions, prevents regression, and enables any developer (human or AI) to contribute without breaking things.

2. Pragmatic Architecture Wins

WordPress + Flutter isn't sexy. It won't win architecture astronaut points. But the system was deployed to production in 32 hours with zero infrastructure provisioning. The best architecture is the one that ships.

3. AI Excels at Consistency, Not Creativity

The AI didn't design the production flow logic or invent the stock calculation algorithm. Those required domain understanding from a human. What AI did brilliantly was apply those decisions consistently across 35 endpoints, 37 Dart files, and 16 database tables without drift.

4. Debug Cycles Compress, Not Disappear

AI doesn't eliminate bugs. It compresses the time between encountering a bug and fixing it. The context window is the debugger — the AI already knows every file, every contract, every constraint. What used to be "search → understand → fix" becomes just "fix."

5. The 80/20 of SDLC Stages

Stage	Time Without AI	Time With AI	Compression
Requirements/Spec	2-3 days	1 hour	20-40x
Schema Design	1 day	1 hour	8-16x
Backend (35 endpoints)	2-3 weeks	6 hours	20-30x
Frontend (37 files)	3-4 weeks	12 hours	25-35x
Testing/Debug	1 week	4 hours	15-20x
Deployment	1 day	30 min	16x
Total	**6-8 weeks**	**~32 hours**	**~25x**

These numbers are real, not hypothetical. The git log proves the timeline.

Conclusion

This project isn't a demo. It's a production system managing real dairy operations with real money. It was built in 32 hours — not because corners were cut, but because AI eliminated the mechanical overhead that dominates traditional development.

The remaining trade-offs are deliberate prioritisation decisions, not oversights — and the system continues to improve. Testing went from 4 files to 12 (52 tests), CSV export was added across all reports, and the codebase passes static analysis with zero issues. The AI-assisted approach didn't just accelerate initial development — it changed the economics of iteration. When you can go from idea to production in a weekend, you can afford to ship, learn, and improve instead of planning, planning, and planning.

The future of software development isn't AI replacing developers. It's developers with AI doing in hours what used to take weeks — and spending the time saved on the things that actually matter: understanding the domain, talking to users, and making the system better.

Built with Claude Code (Anthropic) + Flutter + WordPress. Total codebase: ~10,700 lines across PHP and Dart. 52 tests. Zero analyzer warnings. Time to production: 32 hours.